EE656: ARTIFICIAL INTELLIGENCE, MACHINE LEARNING, DEEP LEARNING & ITS APPLICATIONS

Intelligent Condition Based Monitoring Using Acoustic Signals For Air Compressors

Group - 01

Presented By:

240256 - Bala Swetha
240280 - Boddupally Utham Teja Sree
240384 - Eragandula Neha Sri
241208 - Yashjeet Singh
241196 - Yash Jaiswal

Contents

Introduction

Research Problem and Industrial Context

Automated fault recognition is critical for preventing costly downtime and safety hazards in industrial rotating machinery. Reciprocating air compressors, essential for pneumatic tools and process gas delivery, rely on valves that experience significant wear over time.

Valve damage can lead to:
- Production inefficiencies and unplanned shutdowns
- Hazardous operating conditions
- Substantial economic losses

Introduction

Condition-Based Maintenance (CBM) Approch

CBM represents the most cost-effective maintenance strategy among preventive, breakdown, and condition-based approaches.

Key advantages include:
- Proactive fault detection before catastrophic failure
- Continuous monitoring of critical parameters (vibration, acoustics, temperature, pressure)
- Reduced unnecessary maintenance and minimized downtime
- Optimal scheduling of maintenance activities

Introduction

Acoustic Emission (AE) Monitoring Advantages

AE sensors capture high-frequency stress waves from material defects, offering superior capabilities:

- Insensitive to structural resonances and mechanical background noise
- High sensitivity to incipient faults for early detection
- Effective trending parameters for reliable damage monitoring
- Non-invasive deployment requiring minimal equipment modification

Fault Diagnosis Workflow

6 Steps

Data Acquisition:
Collect raw acoustic data from multiple sensor locations (.wav files)

Sensitive Position Analysis (SPA):
Determine optimal sensor placement for maximum fault signal detection

Preprocessing:
Denoise, filter, and normalize signals to enhance feature quality

Fault Diagnosis Workflow

6 Steps

Feature Extraction:
Compute descriptors across multiple domains: Time, Frequency & Wavelet

Dimensionality Reduction:
Apply PCA, Mutual Information, ICA, or Bhattacharyya distance

Classification:
Use machine learning algorithms (SVM, ANN, LDA) for health state identification

Pre Processing Strategy

Filtering

Removes undesired frequency components from the signal.

High-Pass FIR Filter (Fan Filter):
- Cutoff frequency: 400 Hz
- Eliminates low-frequency fan noise.

Butterworth Low-Pass Filter:
- Order: 18, Cutoff frequency: 12 kHz
- Removes high-frequency environmental noise.

Pre Processing Strategy

Clipping

- Extracts the most stable segment from each 5-second recording.
- Divide into 9 overlapping 1-second segments (50% overlap).
- Compute standard deviation for each segment.
- Select segment with lowest standard deviation for further analysis.

$$\mathbf{X}_{\mathbf{smoothed}}(i) = \frac{\sum_{l=i-q}^{i+q} |\mathbf{x}(l)|}{2q+1}.$$

Pre Processing Strategy

Normalisation

- Ensures consistent scaling across all recordings.
  - Traditional max-min normalization is sensitive to outliers.
  - Modified max-min normalization:
    - Sort values, discard extreme 0.025% at both ends.
    - Use remaining max and min for scaling.
    - Retains amplitude dynamics while reducing outlier impact.

$$\mathbf{X}_{\mathbf{normalized}}(i) = a + \frac{(\mathbf{x}(i) - X_{\min})\,(b-a)}{X_{\max} - X_{\min}}$$

$$where,\ \mathrm{a} = -1 \text{ and } \mathrm{b} = 1. \quad (13)$$

Pre Processing Strategy

Normalisation

Benefits

- Removes irrelevant frequencies and noise.
- Selects clean, consistent signal segments.
- Minimizes outlier influence.
- Provides robust, comparable data for further analysis (e.g., fault detection, condition monitoring).

Pre Processing Strategy

Normalisation

Normalization Algorithm

•Given vector x having l sample values, go through all data samples to find the maximum, and minimum values amongst them, and term them as data_max, and data_min respectively.

•Divide the entire range of sample values into 10,000 equal bins with bin width of eps, where eps is given by

$$eps = \frac{data\_max - data\_min}{10000}. \qquad (14)$$

Pre Processing Strategy

Normalisation

A vector named count is formed where each element of the vector represents a counter variable for each bin. As done in the case of a histogram, go through all sampled values, and assign each sample to the appropriate bin it belongs to. Whenever a new sample is assigned to any of the bins, the corresponding counter variable is incremented by 1.
The bin to which the sample belongs to is not searched; instead it is calculated as follows. In (15), function floor() outputs the nearest integer value below the input.

Pre Processing Strategy

Normalisation

$$
\left.
\begin{array}{l}
\text{if } \mathbf{x}(i) = data\_max \\
\quad bin\_num \leftarrow 10000 \\
\text{else} \\
\quad bin\_num \leftarrow \text{floor}\left(\frac{\mathbf{x}(i) - data\_min}{eps}\right) + 1 \\
\quad \mathbf{count}\left(bin\_num\right) \leftarrow \mathbf{count}\left(bin\_num\right) + 1 \\
\text{end}
\end{array}
\right\}. \quad (15)
$$

Pre Processing Strategy

Normalisation

Find the number of samples to be rejected on extreme ends of the histogram, and assign the value to the variable named reject_num. **reject_num←N×0.00025.**

Find the value of $X_{\min}$, by using the psuedocode given below.

$$
\begin{aligned}
&k = 0\,;\ \ total\_counted = 0\,;\\
&\text{while } total\_counted < reject\_num\\
&\quad k = k + 1;\\
&\quad total\_counted = total\_counted + \mathbf{count}(k)\,;\\
&end\\
&X_{\min} = data\_min + (k-1) \times eps.
\end{aligned}
$$

Pre Processing Strategy

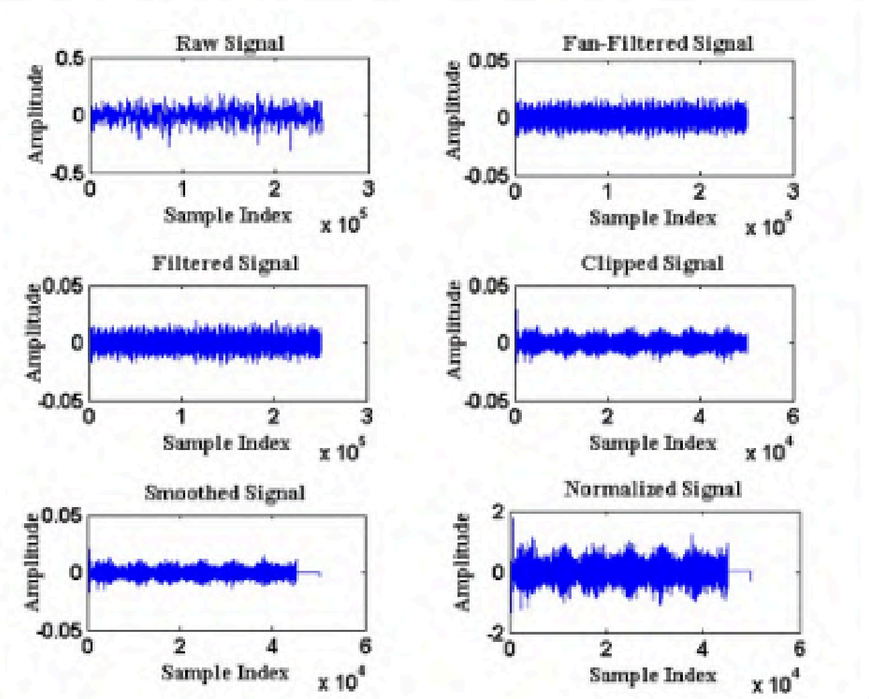
Normalisation

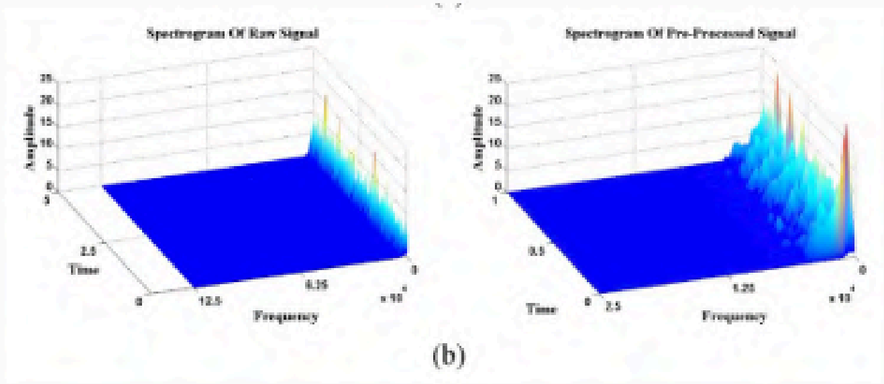
This step is similar to Step 5. Find the value of $X_{\max}$ by using the pseudocode given below.

$k = 10001\,;\ \ total\_counted = 0\,;$
while $total\_counted < reject\_num$
    $k = k - 1;$
    $total\_counted = total\_counted + \mathbf{count}(k)\,;$
end
$X_{\max} = data\_max - (k-1) \times eps.$

- Normalize input vector x with the found Xmin and Xmax values using **(13)**.

Pre Processing Results





Spectrogram plots of signal before and after pre-processing.

Feature Extraction

Converting Raw Signals into Features

Feature extraction is the most critical step in condition-based monitoring. The goal is to convert raw acoustic signals, which by themselves are not very informative, into a set of numerical features that represent the signal's behavior. These features become inputs for classification models. To capture the diverse characteristics of the signals, features are extracted from the time domain, frequency domain, and time-frequency domain.

This approach ensures that both steady-state and transient components are captured. In total, we compute 286 features, including advanced transforms like DCT and STFT, ensuring a comprehensive signal representation.

Feature Extraction

Time Domain Features (8 Features)

Key features

- ***Absolute Mean:*** Average magnitude of the signal, ignoring sign. Indicates overall signal activity.
- ***Peak Value:*** The highest point in the waveform, often a sign of sudden impact.
- ***RMS (Root Mean Square):*** Represents the signal's energy content, sensitive to large amplitudes.
- ***Variance:*** Measures how spread out the values are from the mean.
- ***Skewness:*** Indicates whether the waveform is symmetric or not.
- ***Kurtosis:*** Measures the sharpness or peaky nature of the waveform; high kurtosis may indicate impulsive events.
- ***Crest Factor:*** Ratio of peak value to RMS, useful in identifying high peaks in low-energy signals.
- ***Shape Factor:*** Ratio of RMS to absolute mean; reflects waveform shape characteristics.

Feature Extraction

Frequency Domain Features (8 Features)

Uses Fast Fourier Transform (FFT)
FFT is applied to convert the time-domain signal into its frequency representation. This helps in identifying periodic behavior and harmonics often caused by mechanical faults.

Analyzes frequency content of the signal
Many faults exhibit frequency signatures. For example, imbalance or misalignment produces energy at specific rotational frequencies. These are only visible in the frequency spectrum.

Frequency spectrum divided into 8 bins
To simplify analysis, the frequency spectrum is divided into 8 equally spaced frequency bins. This gives a coarse view of energy distribution across the spectrum.

Energy from each bin used as a feature
The total energy in each bin is calculated (sum of squared FFT amplitudes) and used as a feature. This captures how energy shifts across frequency bands during fault conditions.

Feature Extraction

Time-Frequency Domain Features

Handles non-stationary signals

In many real-world applications, signals are non-stationary, meaning their frequency components change over time. Time-frequency features help track these changes.

Wavelet-based methods used

Wavelets allow simultaneous analysis of both time and frequency. They are particularly effective at detecting transient phenomena like sudden faults.

Morlet Wavelet Transform (MWT) - 7 Features

This uses a Gaussian-modulated sine wave to analyze local frequency content. Features such as entropy, zero-crossing rate, and statistical moments (mean, variance, skewness, kurtosis) are extracted from the wavelet coefficients.

Feature Extraction

Time-Frequency Domain Features

Discrete Wavelet Transform (DWT) - 9 Features

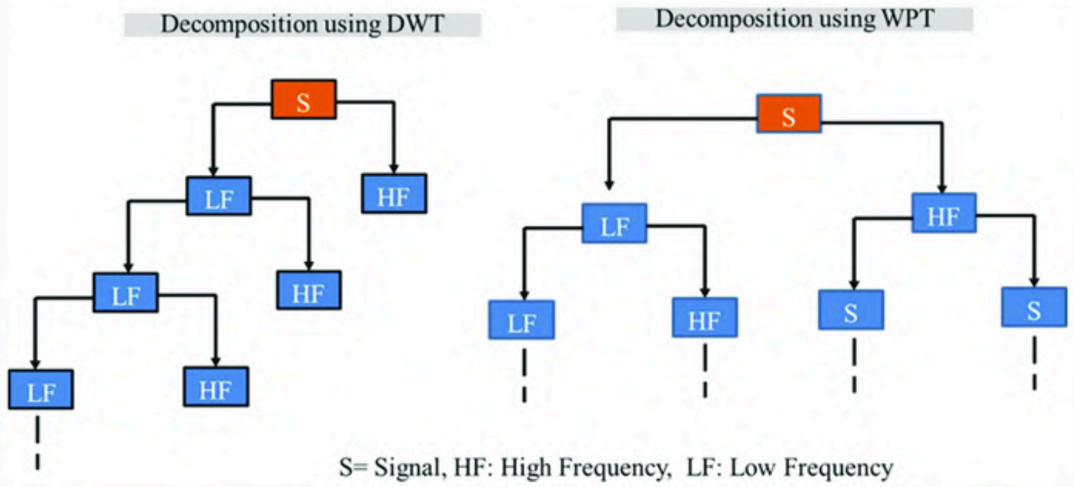
DWT decomposes the signal into approximation and detail coefficients using filters (e.g., Daubechies wavelets). Features like mean, standard deviation, and energy of these coefficients are computed.

Wavelet Packet Transform (WPT) -254 Features

WPT performs a full binary tree decomposition using both low-pass and high-pass filters recursively. The energy of each node is computed, offering a detailed multilevel frequency analysis.

Feature Extraction

DWT vs WPT



Comparison of tree structures: DWT only decomposes low-frequency parts, while WPT splits both LF and HF at each level.

Hence Wavelet Packet Transform extracts more features (256) compared to Discreet Wavelet Transform (9)

Feature Selection

Principal Component Analysis (PCA)

Reduces dimensionality using variance
PCA identifies directions (principal components) where the data varies the most and uses those to represent the dataset in a lower-dimensional space.

Transforms data into new uncorrelated components
It converts original correlated features into orthogonal (uncorrelated) axes, simplifying data structure and improving model training.

Feature Selection

Principal Component Analysis (PCA)

Retains principal components with highest variance

The top principal components are selected based on their variance (eigenvalues). These capture the most significant aspects of the original data.

Projects data into lower-dimensional space

Each sample is projected onto the new component axes, reducing dimensionality while preserving structure. This transformation improves computational efficiency and reduces noise.

Feature Selection

Mutual Information and its Variants

MI measures relevance to class label
Mutual Information quantifies how much a feature contributes to predicting the class label. Higher MI means the feature carries more useful information.

MIFS: relevance minus redundancy
MIFS selects features with high relevance but penalizes those that are redundant with already selected features using a fixed β parameter.

mRMR: dynamically adjusts redundancy factor
mRMR improves MIFS by dynamically adjusting β based on the number of features already selected. This balances relevance and redundancy adaptively.

Feature Selection

Mutual Information and its Variants

NMIFS: normalized MI to remove scale effects

NMIFS normalizes MI values to avoid bias due to scale or range differences across features, ensuring fair comparison.

MIFS-U: uniform info assumption for simplicity

MIFS-U assumes uniform information distribution across features to simplify computation. Though approximate, it is efficient for large-scale problems.

Feature Selection

Bhattacharyya Distance (BD)

Measures class separability using distributions

BD computes how distinguishable two probability distributions are. It considers both the mean and variance of feature values for each class.

Considers mean and covariance of each class

The BD metric uses statistical parameters of class distributions to determine how well a feature separates the classes

Feature Selection

Bhattacharyya Distance (BD)

Selects features with highest BD

The feature that achieves the highest average Bhattacharyya distance from other features is chosen first, as it offers maximum class separability.

Iteratively adds features maximizing mean BD from selected set

In each round, the feature with the highest average BD from the already selected features is added. This process continues until the desired number of features is reached, ensuring the final set is highly discriminative.

Classification

Classification is a machine learning task where the goal is to assign a category or class label to a given input sample based on its features. In the context of fault diagnosis, the input features are extracted from acoustic signals, and the labels indicate the fault type (e.g., valve fault, piston fault, or healthy).

The classification model is first trained on a labeled dataset—where the relationship between input features and fault types is known. Once trained, the model can predict the class of new, unseen samples by applying the learned patterns. This generalization capability is critical in real-time fault detection.

Classification

Several algorithms are commonly employed for classification tasks:

Decision Trees: Simple, interpretable models based on if-else rules.
K-Nearest Neighbor (KNN): Assigns class based on proximity to neighbors in feature space.
Bayes Classifier: Uses probability theory and Bayes' theorem.
Artificial Neural Networks (ANNs): Model complex relationships via multi-layer structures.
Support Vector Machines (SVM): Finds optimal decision boundaries with strong theoretical guarantees.

For industrial applications like air compressor fault detection, models must not only be accurate but also generalize well to unseen conditions. **SVM** is preferred because of its strong theoretical backing, and ability to handle high-dimensional data effectively.

Classification

SVM – Binary Classification

SVM finds a hyperplane that maximizes margin

SVM constructs a decision boundary (hyperplane) that separates data points of two classes. It chooses the hyperplane that maintains the maximum possible distance (margin) from the nearest data points of both classes, known as support vectors. A larger margin is associated with better generalization.

Margin maximization leads to low VC dimension

The VC (Vapnik–Chervonenkis) dimension measures the capacity of a classifier. SVM's focus on maximum margin implicitly reduces the VC dimension, minimizing the risk of overfitting. This theoretical property makes SVM especially suitable for problems with limited or noisy data.

SVM – Binary Classification

SVM works in both linear and non-linear cases

Initially, SVM was designed for linear separation. However, with the kernel trick, SVMs can also handle non-linear separable data by implicitly mapping it into higher-dimensional space, where a linear separator can be found. This makes it powerful for complex decision boundaries.

Real-time performance requires generalization

In online fault monitoring systems, incoming data streams are continuously classified. Therefore, the classifier must generalize well to changing conditions, minor variations, and previously unseen faults. SVM provides this robustness due to its max-margin formulation and regularization capabilities.

Kernel & Soft Margin SVM

In many real-world datasets, a linear separator does not exist in the original input space. The kernel trick allows SVM to operate in a high-dimensional space without explicitly computing the transformation. It replaces the dot product with a kernel function, enabling efficient handling of complex boundaries

Common kernel types used in SVM

1. **Linear kernel:** Used when data is linearly separable.
2. **Polynomial kernel:** Captures curved boundaries using polynomial functions.
3. **Quadratic kernel:** A special case of the polynomial kernel (degree 2).
4. **Radial Basis Function (RBF) kernel:** Maps data into infinite-dimensional space and is widely used for non-linear data due to its flexibility and effectiveness

Kernel & Soft Margin SVM

In practical applications, data is often noisy and not perfectly separable. Soft margin SVM introduces slack variables ξ to allow some margin violations. This flexibility improves real-world applicability.

The cost parameter CCC balances margin width and classification error. A large CCC penalizes misclassifications heavily, possibly leading to overfitting. A smaller CCC allows a wider margin with more tolerance for error, promoting better generalization.

Mathematical Formulation of SVM

Primal Form (with slack variables)

the optimization objective of soft-margin SVM is:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \qquad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

This objective tries to minimize the model complexity (through ||w||^2) while penalizing misclassifications (through ξ).

Dual Form (with Lagrange multipliers)

Instead of solving the primal directly, we use the dual formulation:

$$\max_{\alpha} \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i,j}\alpha_i\alpha_j y_i y_j K(x_i, x_j) \qquad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{n}\alpha_i y_i = 0$$

Final decision function

The final prediction for a new input x is:

$$f(x) = \sum_{i=1}^{n}\alpha_i y_i K(x_i, x) + b$$

This decision function computes a weighted sum over support vectors using the kernel.

Classifier Comparison

Classifier	Accuracy (%)
SVC (RBF, DDAG)	99.4
MLP	99.2
Random Forest	97.7
KNN ($k = 5$)	94.6
Decision Tree	92.2

The SVC came out on top, with the MLP a close second. Given the SVC's strong accuracy, solid theoretical grounding, and far lower training cost than the MLP, we chose it (with an RBF kernel) as our primary classifier for the full sweep of experiments below.

Multiclass SVM Strategies

Fault diagnosis is a multiclass classification problem

In air compressors or industrial systems, faults are not binary. Multiple fault conditions must be recognized: valve failure, piston fault, healthy state, etc. SVM, which is inherently binary, must be extended to handle these multiclass scenarios.

1. One Against One (OAO) – Pairwise strategy

For nnn classes, n(n-1)2\frac{n(n-1)}{2}2n(n-1) binary classifiers are trained—each to distinguish between a pair of classes. During prediction, all classifiers vote, and the class with the highest number of votes is selected. OAO is computationally efficient during testing and handles imbalanced datasets well.

Multiclass SVM Strategies

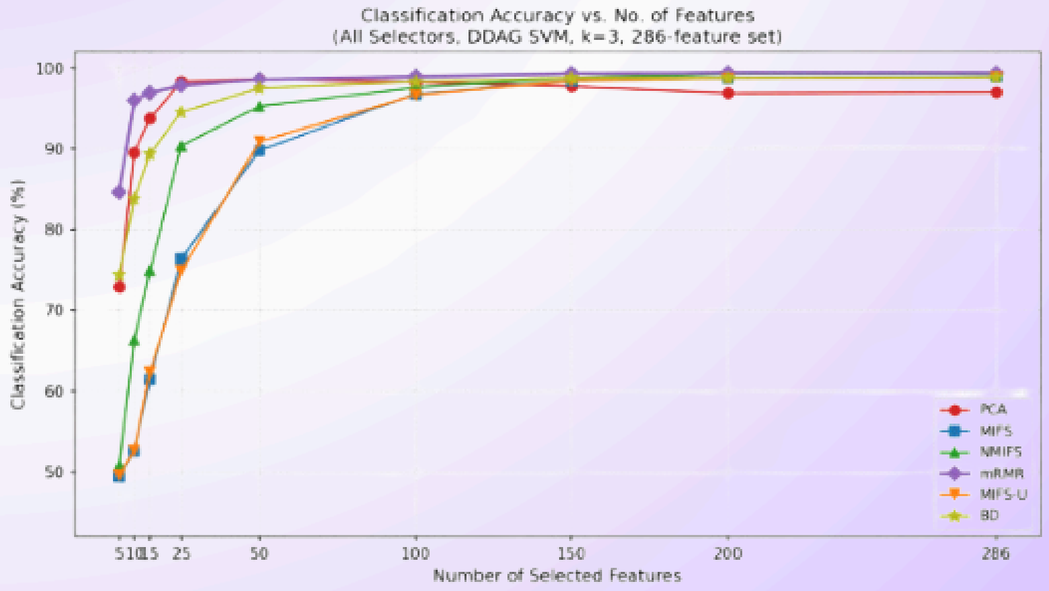
2. One Against All (OAA) – Class-vs-Rest strategy

Here, nnn classifiers are trained, each distinguishing one class from all others. The classifier that gives the highest decision score during prediction determines the output class. While simple to implement, OAA may suffer when class imbalance is high because each classifier sees multiple negative examples.

3. Decision Directed Acyclic Graph (DDAG)

DDAG arranges binary classifiers in a graph. At each node, a classifier eliminates one class, and the process continues until one class remains. DDAG reduces the number of decisions needed per prediction and is efficient during inference.

Method	5	10	15	25	50	100	150	200	286
PCA	72.89	89.56	93.78	98.28	98.67	98.22	97.72	96.89	97.00
MIFS	49.39	52.56	61.44	76.39	89.78	96.78	98.39	98.72	98.83
NMIFS	50.78	66.33	74.94	90.39	95.28	97.56	98.78	**99.28**	99.25
mRMR	**84.61**	**96.00**	**96.94**	**97.83**	**98.56**	**98.94**	**99.33**	99.44	**99.42**
MIFS-U	49.56	52.61	62.39	74.89	90.89	96.67	98.39	98.72	98.83
BD	74.39	83.78	89.39	94.56	97.50	98.28	98.78	98.78	98.83



Feature Selection Comparison

Our first sweep compared all six feature-selection methods on the 286-feature base set, using a DDAG SVM with an RBF kernel at k = 3, across feature-count budgets from 5 to the full 286

Table 5: Classification accuracy (%): OAO vs. OAA vs. DDAG. Selector: mRMR. Classifier: C-SVM, RBF kernel. $k = 3$.

Decomposition	5	10	15	25	50	100	286	Total time (min)
OAO	84.78	96.00	96.78	97.11	98.83	98.94	99.06	16.41
OAA	84.78	96.11	**97.11**	**98.50**	**99.22**	**99.17**	99.06	11.41
DDAG	**84.78**	**96.11**	96.94	97.83	98.56	98.94	99.06	**3.35**

Table 6: Per-run wall-clock training time (minutes) at each feature count.

Decomposition	5	10	15	25	50	100	286 (cached)
OAO	1.25	1.13	1.25	2.12	2.66	8.00	0.00
OAA	0.79	0.83	0.98	1.44	1.82	5.55	0.00
DDAG	**0.35**	**0.36**	**0.40**	**0.45**	**0.55**	**1.25**	0.00

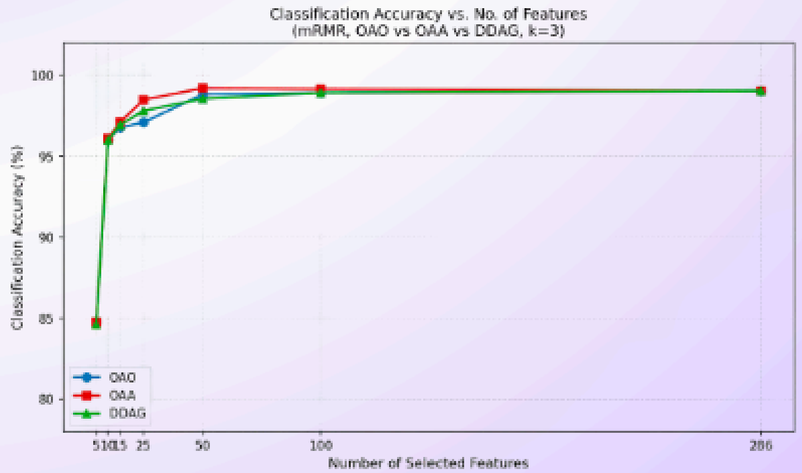


Figure 4: Accuracy vs. features for OAO, OAA, and DDAG (mRMR selector, $k = 3$).

Multiclass Decomposition Strategy

Our second sweep fixed the feature selector to mRMR (from earlier result) and k = 3, and compared the three multiclass decomposition strategies — OAO, OAA, and DDAG both for accuracy and for wall-clock training time

Table 7: Number of mRMR-selected features per domain, 286-feature set (averaged across 3 folds).

No. features	TD	FFT	MWT	DWT	WPT	Total
5	0	1	1	0	3	5
10	0	1	1	0	8	10
15	0	1	1	0	13	15
25	0	1	1	0	23	25
50	0	1	2	0	47	50
75	0	1	4	0	70	75
100	1	2	5	2	90	100
286	8	8	7	9	254	286

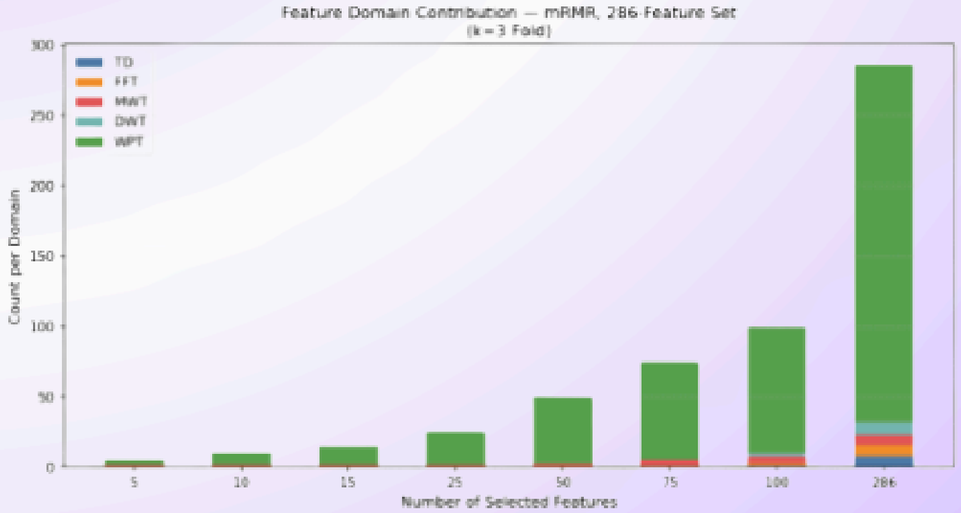


Figure 5: Domain contribution of mRMR-selected features, 286-feature set.

Which Domains Does the Selector Actually Use?

To understand why mRMR performs so well, we tracked which feature domain each selected feature came from, at every budget, averaged across the three folds. Table 7 shows this for the 286-feature base set.

Table 8: Classification accuracy (%) vs. number of selected features, 629-feature extended set. Classifier: DDAG SVM, RBF kernel, $k = 3$. BD omitted (run not completed). Bold = best at that feature count.

Method	5	10	15	25	50	100	300	500	629
PCA	72.89	89.56	93.78	98.28	98.67	98.22	96.89	95.67	96.58
MIFS	49.39	52.56	61.44	76.39	89.78	96.78	98.72	99.11	99.17
NMIFS	50.78	66.33	74.94	90.39	95.28	97.56	99.28	99.11	99.17
mRMR	**84.61**	**96.00**	**96.94**	**97.83**	**98.56**	**98.94**	**99.44**	99.11	**99.17**
MIFS-U	49.56	52.61	62.39	74.89	90.89	96.67	98.72	**99.11**	**99.17**

The single best result anywhere in our study is **mRMR** at **300 selected features** from the extended set, at **99.44%** accuracy.

Table 9: Head-to-head comparison: 286-feature base set vs. 629-feature extended set, mRMR + DDAG SVM, $k = 3$.

No. of features	286-set acc. (%)	629-set acc. (%)	Δ (pp)
5	84.61	84.61	0.00
10	96.00	96.00	0.00
15	96.94	96.94	0.00
25	97.83	97.83	0.00
50	98.56	98.56	0.00
100	98.94	98.94	0.00
286 / 300	99.42	99.44	+0.02

Does the Extended 629-Feature Set Help?

We tested whether the 629-feature extended set which adds DCT, STFT, autocorrelation, UMT, convolution-with-sinusoid, and four quadratic time–frequency distributions (WVD, PWVD, CWD, BJD) on top of the 286 base features actually improves accuracy.

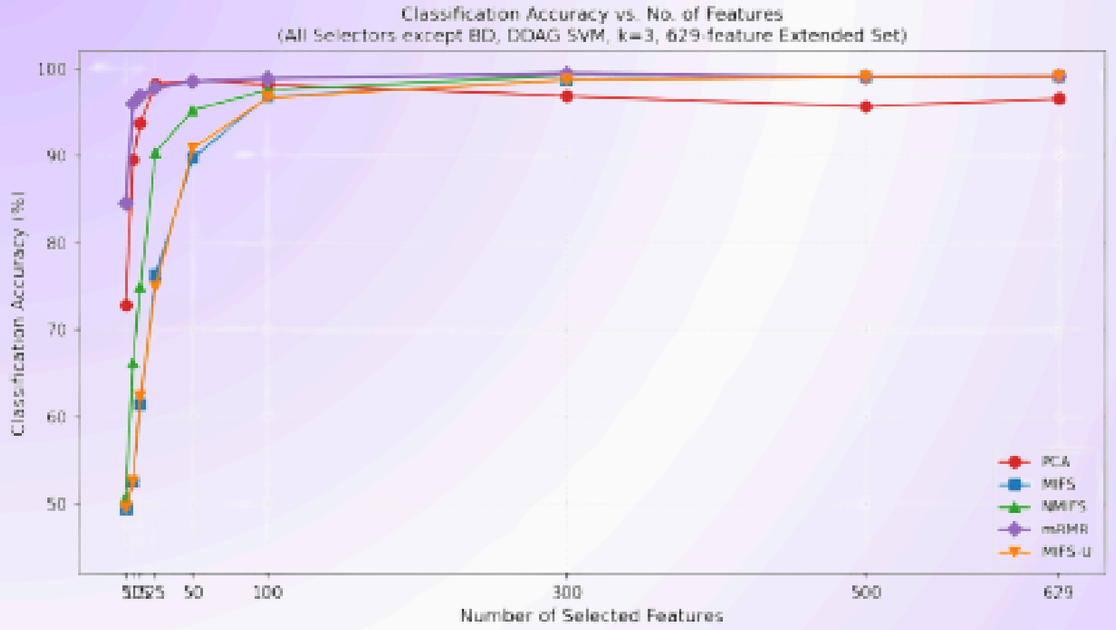


Figure 6: Accuracy vs. selected features, 629-feature extended set (BD omitted).

Therefore, tripling the feature vocabulary buys essentially nothing. Below 100 features the 286-set and 629-set accuracies are identical to two decimal places, because the selectors simply never reach into the extended transforms at those budgets, there is nothing new to gain.

Does the Extended 629-Feature Set Help?

We tested whether the 629-feature extended set which adds DCT, STFT, autocorrelation, UMT, convolution-with-sinusoid, and four quadratic time–frequency distributions (WVD, PWVD, CWD, BJD) on top of the 286 base features actually improves accuracy.

Feature Selection Conclusion

Putting everything together, the configuration we recommend and the one that produced our best validated result is **mRMR feature selection on the 629-feature extended set, keeping 300 features, with a DDAG-decomposed RBF SVM at k = 3 cross-validation**, reaching **99.44% accuracy.**

If the extra 0.02 percentage points are not worth the cost of building the extended feature set, the practical recommendation is simpler still: **mRMR on the 286-feature base set, 286 features, DDAG SVM, at 99.42%**, with no extended-transform computation required at all.

Either way, the headline conclusion stands as our central finding: **mRMR** is the selector that matters, and once it is in the loop, the choice of multiclass strategy and the size of the feature vocabulary are second-order concerns.

Beyond the SVC – Exploring Gradient-Boosted Trees

Extension: Gradient-Boosted Trees (XGBoost)

- The Support Vector Classifier (SVC) serves as our primary model, yielding excellent baseline accuracy.
- We explored XGBoost to see if a fundamentally different model family could match or complement the SVC.
- Goal: Determine if combining a hyperplane-based model (SVC) with a decision-tree-based model (XGBoost) pushes accuracy beyond what either achieves alone.

- Native Multiclass Support: Unlike SVC, which requires decomposition strategies (OAO, OAA, DDAG), XGBoost handles all eight compressor fault classes directly via a multi:softprob objective.
- Robust to High Dimensionality: Effectively handles the massive 286-feature set without strictly requiring explicit feature selection beforehand.
- Complementary Error Profiles: Tree-based models partition data differently than SVMs, making them ideal candidates for model ensembling.

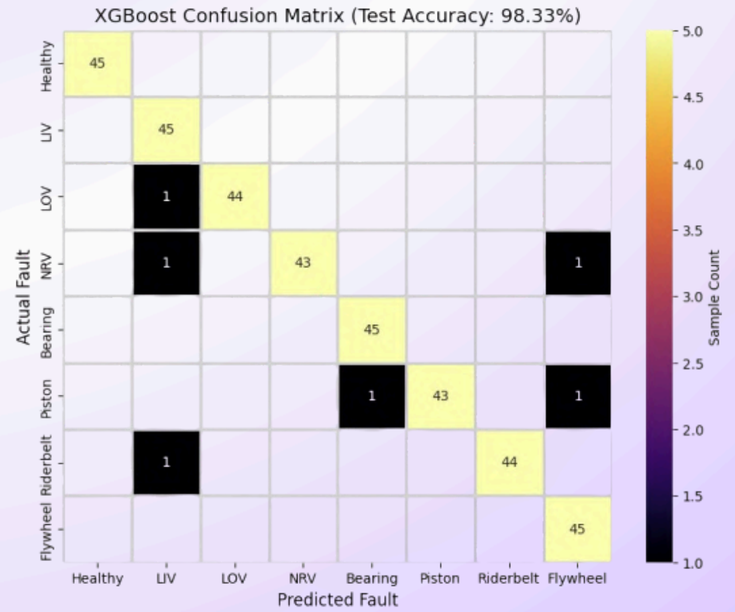
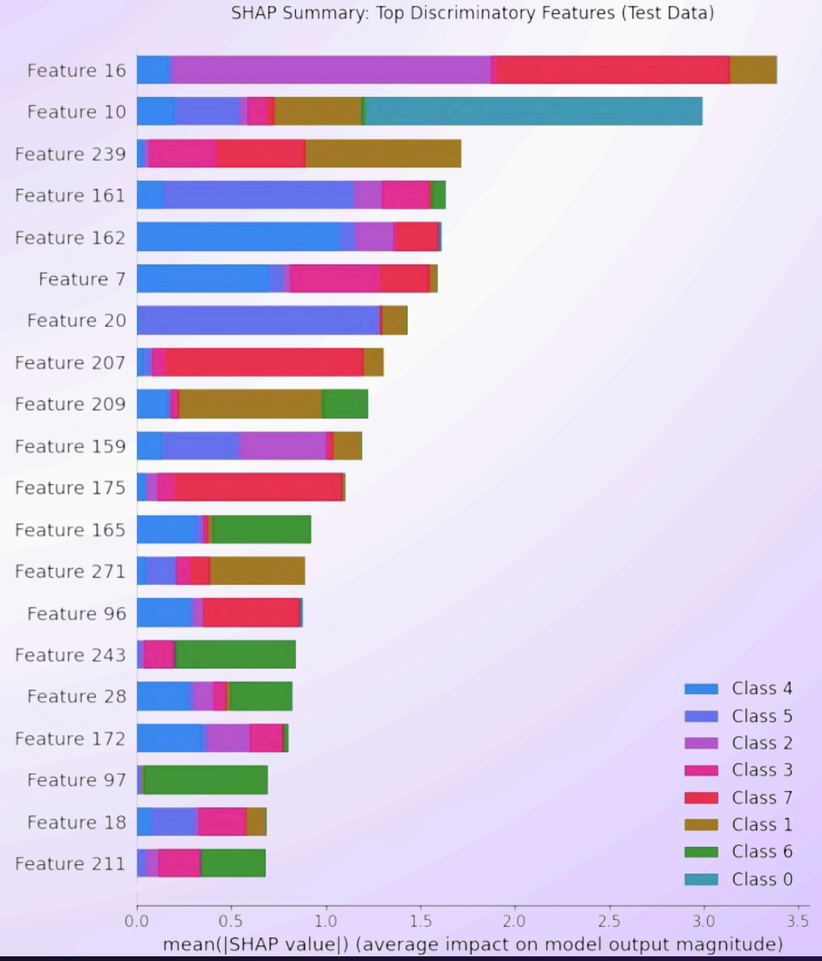


Figure 7: XGBoost confusion matrix on the held-out test set (286-feature base set).

Motivation for XGBoost Implementation

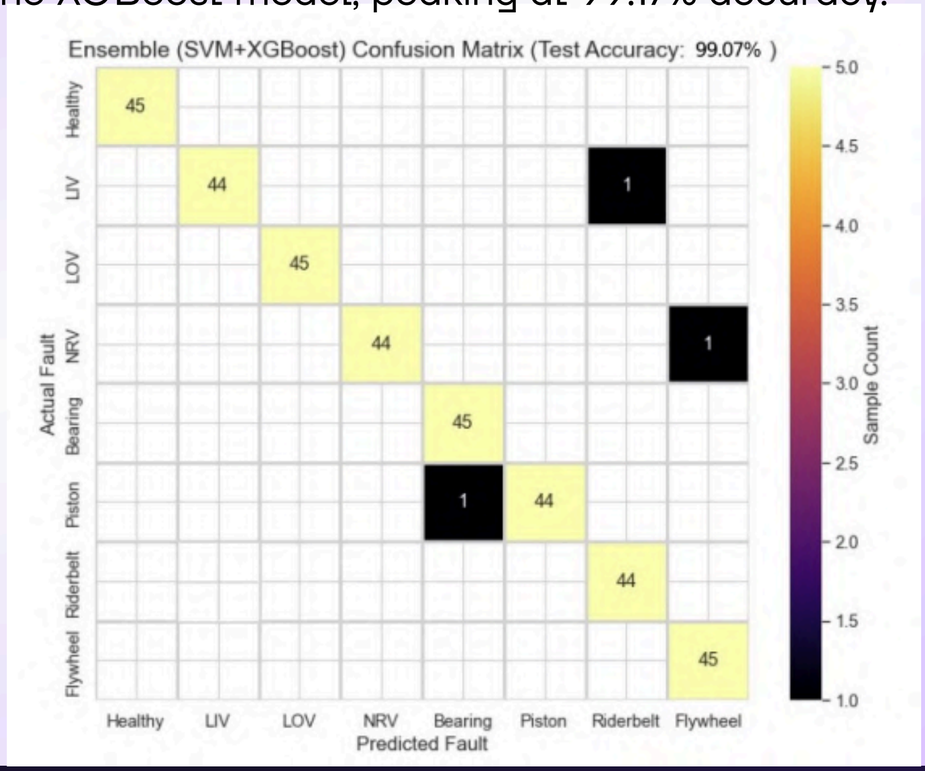
XGBoost vs. SVC Baseline Performance

- SVC (OAO) Baseline: Achieved a near-ceiling accuracy of 99.06%.
- XGBoost Baseline: Reached an accuracy of 98.67%.
- While XGBoost performed exceptionally well, it fell slightly short of the classical SVC approach on this specific acoustic dataset.

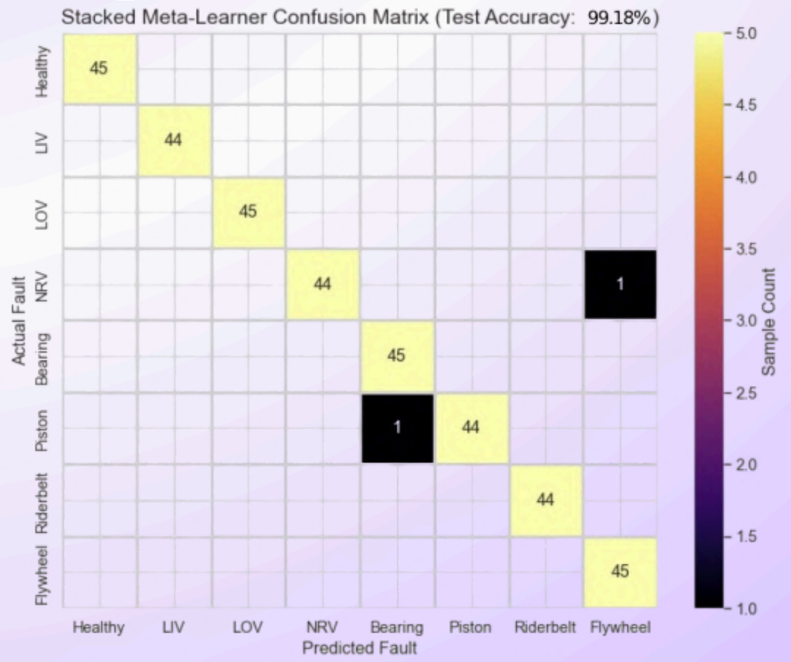


Ensembling: Combining SVC and XGBoost

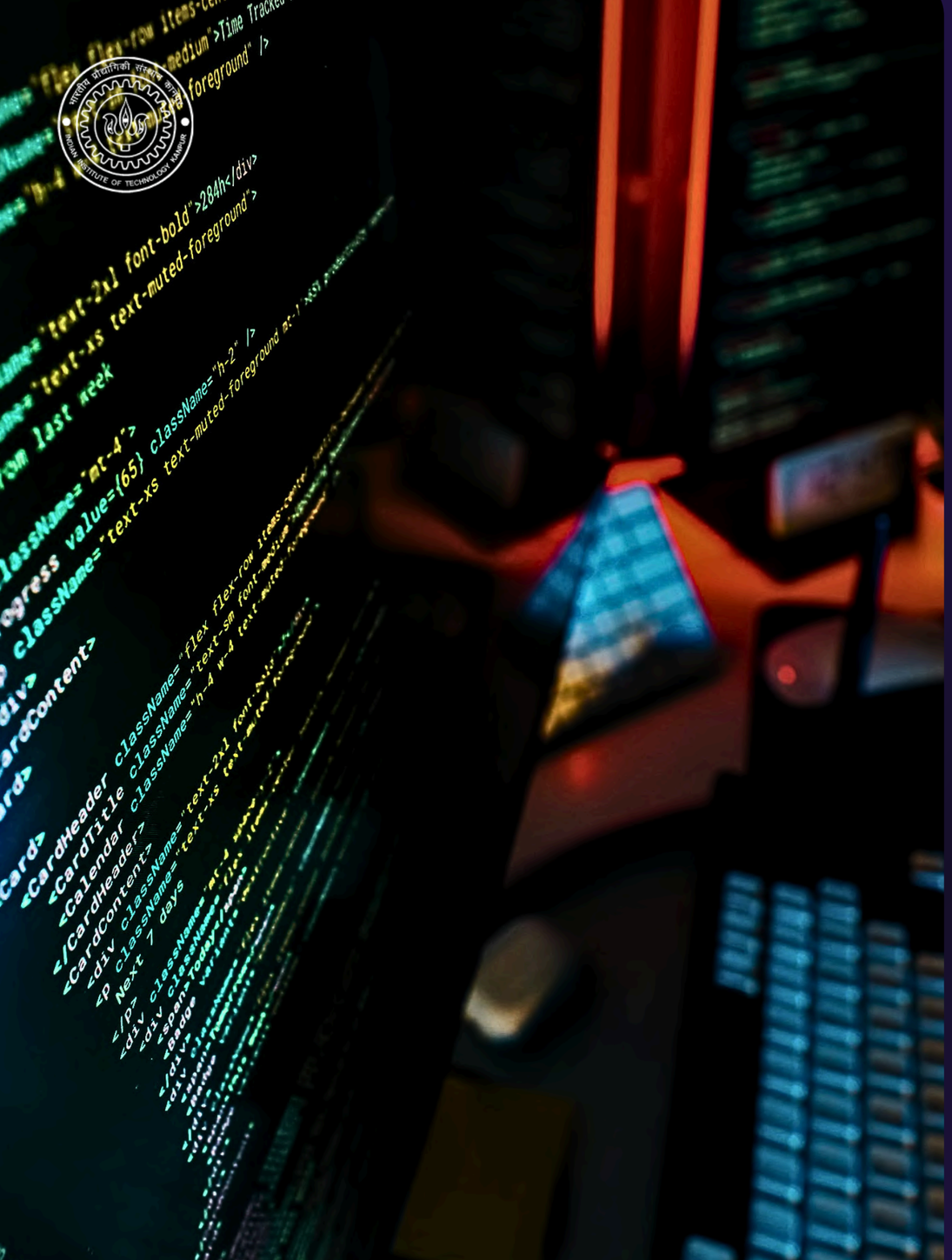
- We combined the predictive power of both models using two ensemble techniques.
- Soft Voting Ensemble: Averages the output probabilities of both models, pushing the overall accuracy to 99.11%.
- Stacking Ensemble: Utilized a Logistic Regression meta-classifier to learn when to trust the SVC versus the XGBoost model, peaking at 99.17% accuracy.



- Ensembling successfully pushed the absolute accuracy limit, gaining a marginal ~0.11% improvement over the baseline SVC.
- The Trade-off: This fractional gain comes at the cost of running two heavy parallel models, drastically increasing inference time and computational overhead.
- Final Verdict: For a deployable, lightweight industrial Condition Based Monitoring (CBM) system, the classical standalone SVC remains the superior engineering choice.



Conclusion: Cost as a First-Class Concern



Results & Conclusion

Results

Model / Strategy	Test Accuracy (%)	Misclassified (of 360)
XGBoost (Optuna-tuned, 286 features)	98.33	6
SVC (mRMR, 286 features, OAO)	98.89	4
Soft-voting ensemble (SVM + XGBoost)	99.07	3
Stacked meta-learner (logistic regression)	**99.18**	**2**

Therefore, **Stacked Meta-Learner (Logistic Regression)** was the best of all with **99.18% accuracy** and only **2 misclasifications.**

Conclusion

In this project we built, ran, and analyzed a complete acoustic fault-diagnosis system for reciprocating air compressors, implementing every stage ourselves in Python: EMD-based sensitive position analysis, a four-step pre-processing routine, multi-domain feature extraction yielding 286 base features (629 in an extended set), six feature-selection methods, and multiclass classification across five models and three decomposition strategies. We backed every major claim with a dedicated, documented experiment rather than a single headline number.

More than the numbers, the project gave us a hands-on appreciation of how signal processing and machine learning combine to solve a real industrial problem — and a concrete demonstration that a thoughtfully engineered classical pipeline, with a carefully chosen and appropriately sized feature set, can match a deep network at a tiny fraction of the cost. The model is only ever as good as the features it is built on and the selector that chooses among them — and that, more than any single algorithm or feature count, is the lesson we will carry forward.

Thankyou

Group - 01